Week-10 Lab Programs

1. Genetic Algorithm to Maximise Number of 1's

AIM

To implement a Genetic Algorithm to maximize the number of 1's in a binary string. The algorithm uses genetic operations such as selection, crossover, and mutation to evolve a population of binary strings over successive generations until the maximum number of 1's is achieved.

Description

A Genetic Algorithm is an optimization technique inspired by natural selection and genetics. In this problem, each individual is represented as a binary string, and its fitness is calculated by counting the number of 1's. Selection, crossover, and mutation are applied repeatedly to improve the population until the optimal solution is obtained.

Code

```
import random

N = 8
POP_SIZE = 10

def fitness(chromosome):
    return chromosome.count('1')

def crossover(p1, p2):
    point = random.randint(1, N - 1)
    return p1[:point] + p2[point:]

def mutate(chromosome):
    i = random.randint(0, N - 1)
    bits = list(chromosome)
    bits[i] = '1' if bits[i] == '0' else '0'
    return ''.join(bits)

population = [
    ''.join(random.choice('01') for _ in range(N))
    for _ in range(POP_SIZE)
]

generation = 0

while True:
```

```
    population.sort(key=fitness, reverse=True)

    if fitness(population[0]) == N:
        break

    new_population = population[:2]

    while len(new_population) < POP_SIZE:
        p1, p2 = random.choices(population[:5], k=2)
        child = crossover(p1, p2)

        if random.random() < 0.2:
            child = mutate(child)

        new_population.append(child)

    population = new_population
    generation += 1

print("Best Chromosome:", population[0])
print("Fitness:", fitness(population[0]))
print("Generations:", generation)
```

Output

```
Best Chromosome: 11111111
Fitness: 8
Generations: 3
```

---

2. 8 Queens

AIM

To solve the 8-Queens Problem using a Genetic Algorithm by placing eight queens on an 8×8 chessboard such that no two queens attack each other. The algorithm uses selection, crossover, and mutation to evolve possible solutions and find an arrangement with no attacking pairs of queens.

Description

The 8-Queens Problem is a classic Artificial Intelligence problem that involves placing eight queens on a chessboard without conflicts. In a Genetic Algorithm, each chromosome represents a possible arrangement of queens, and its fitness is calculated based on the number of non-attacking pairs. Selection, crossover, and mutation are applied repeatedly until a valid solution is found.

Code

```
import random

N = 8
POP_SIZE = 100

def fitness(board):
    score = 0

    for i in range(N):
        for j in range(i + 1, N):
            if abs(board[i] - board[j]) != abs(i - j):
                score += 1

    return score

def crossover(p1, p2):
    point = random.randint(1, N - 1)
    child = p1[:point]
    child += [x for x in p2 if x not in child]
    return child

def mutate(board):
    board = board[:]
    i, j = random.sample(range(N), 2)
    board[i], board[j] = board[j], board[i]
    return board

def print_board(board):
    for row in range(N):
        for col in range(N):
            print("Q" if board[col] == row else ".", end=" ")
        print()

population = [
    random.sample(range(N), N)
    for _ in range(POP_SIZE)
]

generation = 0

while True:
    population.sort(key=fitness, reverse=True)
    best = population[0]

    if fitness(best) == 28:
        break
```

```
    new_population = population[:2]

    while len(new_population) < POP_SIZE:
        p1, p2 = random.choices(population[:20], k=2)
        child = crossover(p1, p2)

        if random.random() < 0.2:
            child = mutate(child)

        new_population.append(child)

    population = new_population
    generation += 1

print("Solution:", best)
print("Fitness Score:", fitness(best))
print("Generations:", generation)
print("\\nChessboard:")
print_board(best)
```

Output

```
Chessboard:
. . . . . . . Q
. . Q . . . . .
Q . . . . . . .
. . . . . Q . .
. Q . . . . . .
. . . . Q . . .
. . . . . . Q .
. . . Q . . . .
```